Search Engine Choices — Design Rationale

Version: 1.0
Date: April 15, 2026
Author: Architecture Team
Parent Doc: banking-application-architecture.md

Table of Contents

1. Why We Need a Dedicated Search Engine
2. Candidates Evaluated
3. Decision: Elasticsearch
4. Elasticsearch vs. Alternatives — Deep Comparison
5. Why Not PostgreSQL Full-Text Search Alone?
6. Elasticsearch Deployment Architecture
7. When to Consider Alternatives in the Future

---

1. Why We Need a Dedicated Search Engine

PostgreSQL is our source of truth, but it falls short for user-facing search:

```
USER TYPES: "amzon grocery jan"
              |         |     |
              |         |     └─ Intent: date filter (January)
              |         └─ Intent: category (groceries)
              └─ Problem: misspelled merchant name

┌──────────────────────────────────────────────────────────────────────┐
│  What PostgreSQL can do:                                             │
│  • ILIKE '%amzon%'     → 0 results (exact substring, no fuzzy)       │
│  • to_tsvector search  → requires exact tokens ("amazon" not "amzon") │
│  • B-tree index scan   → exact match only                            │
│                                                                      │
│  What we NEED:                                                       │
│  • Fuzzy match "amzon" → "Amazon" (Levenshtein distance 1)           │
│  • Synonym expansion "grocery" → also match "groceries", "supermarket"│
│  • Relevance ranking: recent Amazon grocery > old Amazon electronics  │
│  • Autocomplete: "amz" → suggest "Amazon"                            │
│  • Highlighted matches in results                                    │
│  • Aggregations: "45 UPI, 12 NEFT" facets alongside results          │
│  • Sub-200ms response on 100M+ documents                             │
│                                                                      │
│  → Dedicated search engine is required.                              │
└──────────────────────────────────────────────────────────────────────┘
```

2. Candidates Evaluated

Search Engine	Type	License	Backing
Elasticsearch	Distributed search & analytics	SSPL (Server Side Public License)	Elastic NV
OpenSearch	Elasticsearch fork (pre-SSPL)	Apache 2.0	AWS (community)
Apache Solr	Distributed search platform	Apache 2.0	Apache Foundation
Meilisearch	Lightweight search engine	MIT	Meilisearch
Typesense	Lightweight search engine	GPL-3.0	Typesense
PostgreSQL FTS	Built-in full-text search	PostgreSQL License	PostgreSQL Community

3. Decision: Elasticsearch

Chosen: Elasticsearch 8.x

Primary reasons:

1. Most mature ecosystem for application search at banking scale.
2. Rich query DSL covering all our requirements (fuzzy, autocomplete, aggregations, relevance scoring).
3. Proven at scale: handles hundreds of millions of documents with sub-200ms query latency.
4. Best-in-class tooling: Kibana (visualization), Logstash (data pipeline), and native Kubernetes operators.
5. Strong integration with our observability stack (ELK for logging is already planned, so operational knowledge is shared).

---

4. Elasticsearch vs. Alternatives — Deep Comparison

4.1 Elasticsearch vs. OpenSearch

```
┌──────────────────────────────────────────────────────────────────────┐
│              Elasticsearch vs. OpenSearch                            │
│                                                                      │
│  Origin: OpenSearch is a fork of Elasticsearch 7.10 (2021)           │
│  Created by AWS when Elastic changed to SSPL license                 │
│                                                                      │
│  Feature                Elasticsearch 8.x       OpenSearch 2.x       │
│  ────────                ─────────────────       ──────────────      │
│  Query DSL               Identical (legacy)      Identical (forked)  │
│  Vector search (kNN)     Native (dense_vector)   Native (also has it) │
│  Security                X-Pack (built-in)       OpenSearch Security │
│  ML / Inference          Elastic ML              OpenSearch ML       │
│  Aggregations            Identical               Identical           │
│  Managed on AWS          Amazon OpenSearch Svc    Amazon OpenSearch Svc │
│  Managed on GCP/Azure    Elastic Cloud           Not available natively │
│  Kubernetes operator     ECK (official)          OpenSearch operator  │
│  Kibana equivalent       Kibana                  OpenSearch Dashboards │
│  Community size          Larger                  Growing              │
│  License                 SSPL                    Apache 2.0           │
│                                                                      │
│  VERDICT: Both are viable.                                           │
│                                                                      │
│  We chose Elasticsearch because:                                     │
│  • Richer ecosystem (Kibana > OpenSearch Dashboards for visualizations)│
│  • Faster feature velocity (ES 8.x has ESQL, vector search advances) │
│  • ECK (Elastic Cloud on Kubernetes) is more mature                  │
│  • Operational synergy — ELK stack for logging uses same ES cluster  │
│    knowledge, reducing team learning curve                           │
│                                                                      │
│  If AWS-native deployment is required, OpenSearch is the drop-in     │
│  replacement. Query DSL is 98% compatible. Migration cost is low.    │
└──────────────────────────────────────────────────────────────────────┘
```

4.2 Elasticsearch vs. Apache Solr

Dimension	Elasticsearch	Apache Solr
Query DSL	JSON-based, programmatic, composable	XML/URL params, less developer-friendly
Cluster management	Self-contained (no external dependency)	Requires Apache ZooKeeper for SolrCloud
REST API	Native, clean, well-documented	Functional but less ergonomic
Real-time indexing	NRT (near real-time), ~1s visibility	NRT, similar to ES
Scaling	Add nodes, shards auto-rebalance	Requires ZooKeeper coordination
Kubernetes	ECK operator (official, production-grade)	No official K8s operator
Ecosystem	Kibana, Beats, Logstash, APM, ML	Banana (community), limited tooling
Community	Massive (GitHub: 70K+ stars)	Large but smaller than ES
JSON native	First-class JSON document support	XML-oriented, JSON as an afterthought

Verdict: Solr is a capable search engine but Elasticsearch's developer experience, REST API design, Kubernetes integration, and ecosystem make it the stronger choice for a modern microservices architecture. Solr's ZooKeeper dependency adds operational overhead we want to avoid.

4.3 Elasticsearch vs. Meilisearch / Typesense

Dimension	Elasticsearch	Meilisearch / Typesense
Scale	Billions of documents, petabytes	Millions of documents, single-node or small cluster
Query complexity	Full bool/nested/function_score queries	Simple search + filters (limited query DSL)
Aggregations	Full aggregation framework	Basic facets
Custom analyzers	Custom tokenizers, synonyms, edge n-grams	Pre-built, limited customization
Distributed	Sharded, replicated, multi-node	Meilisearch: single-node (cloud has multi). Typesense: basic clustering
Operational maturity	10+ years in production at scale	Younger, less battle-tested
Banking fit	Proven in financial services	Designed for e-commerce/product search

Verdict: Meilisearch and Typesense are excellent for simpler use cases (e-commerce product search, documentation search) where ease of setup matters more than query complexity. For a banking application with:

- 100M+ documents
- Complex multi-field relevance scoring
- Aggregations for faceted filters
- Custom analyzers (synonyms for banking terms)
- Strict operational requirements (HA, backups, monitoring)

Elasticsearch is the appropriate choice. Meilisearch/Typesense would require workarounds for our aggregation and custom analyzer needs.

---

5. Why Not PostgreSQL Full-Text Search Alone?

PostgreSQL has built-in full-text search (`tsvector`, `tsquery`, `GIN` index). We evaluated it seriously.

5.1 What PostgreSQL FTS Can Do

```
-- Create a GIN index for full-text search
CREATE INDEX idx_txn_fts ON transactions
    USING gin(to_tsvector('english', description));

-- Query with ranking
SELECT id, description, amount,
       ts_rank(to_tsvector('english', description), query) AS rank
FROM transactions,
     to_tsquery('english', 'amazon & grocery') AS query
WHERE to_tsvector('english', description) @@ query
  AND account_id = 'ACC-001'
ORDER BY rank DESC
LIMIT 20;
```

5.2 What PostgreSQL FTS Cannot Do (Well)

Requirement	PostgreSQL FTS	Elasticsearch
Fuzzy matching	No native fuzzy. `pg_trgm` extension helps with similarity but no Levenshtein-based fuzzy at query time.	Native: `"fuzziness": "AUTO"` handles 1-2 character typos transparently
Edge n-gram autocomplete	No. Would need a separate materialized table with n-gram tokens.	Native: `edge_ngram` analyzer + prefix queries, sub-50ms
Multi-field boosting	Limited. `ts_rank` works on a single `tsvector` column. Boosting merchant name 5x over description requires manual weight tuning.	Native: `multi_match` with per-field `^boost` values
Synonyms	Thesaurus dictionaries exist but are static files, require PG restart to update.	Synonym filter in analyzer, updateable via API without restart
Highlighting	`ts_headline()` exists but is slow (re-parses document at query time).	Native highlighting, fast (stored in inverted index)
Aggregations alongside results	Requires separate `GROUP BY` query or window functions. Two round trips or complex CTE.	Single query returns results + aggregations in one response
Relevance scoring	`ts_rank` with 4 weight categories (A, B, C, D). Limited customization.	Full BM25 + function_score with decay functions, script scoring, custom boosting
Performance at scale	FTS on a partitioned table with 100M+ rows: 200-500ms even with GIN index. Competes with OLTP workload on the same PG instance.	Dedicated cluster, 50-150ms on same data volume. No impact on transactional PG.
Operational independence	Shares resources (CPU, RAM, I/O) with transactional queries. Heavy search = slow transfers.	Separate cluster. Search load has zero impact on transaction processing.

5.3 Decision: ES for Primary Search, PG FTS as Fallback

```
Normal operation:
  Search query → Elasticsearch (full features)

ES circuit breaker OPEN:
  Search query → PostgreSQL FTS (degraded: no fuzzy, no autocomplete)
  UI shows: "Search features temporarily limited"

PG FTS is the safety net, not the primary path.
```

5.4 Cost of Adding Elasticsearch

Cost	Detail
Infrastructure	3 data nodes + 2 master nodes: ~$2,800/month
Operational	Monitoring (already part of ELK stack), index management (ILM automates lifecycle)
Development	ES client integration, index mapping, consumer for Kafka-to-ES pipeline
Latency	Async indexing adds 2-5 second delay for new documents to be searchable

The cost is justified because:

- Users search hundreds of times per day. Sub-200ms fuzzy search with relevance ranking is a core UX requirement.
- Running heavy search queries on the transactional PostgreSQL instance risks degrading transfer/payment latency — the most critical SLO.
- The ELK stack for logging already introduces Elasticsearch operational knowledge to the team.

---

6. Elasticsearch Deployment Architecture

6.1 Cluster Topology

```
┌──────────────────────────────────────────────────────────────────────┐
│                      ELASTICSEARCH CLUSTER                           │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐               │
│  │ Master Node 1│  │ Master Node 2│  │ Master Node 3│               │
│  │ (dedicated)  │  │ (dedicated)  │  │ (dedicated)  │               │
│  │              │  │              │  │              │               │
│  │ Only cluster │  │ Quorum-based │  │ Prevents     │               │
│  │ state mgmt   │  │ leader elect │  │ split-brain  │               │
│  └──────────────┘  └──────────────┘  └──────────────┘               │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐               │
│  │ Data Node 1  │  │ Data Node 2  │  │ Data Node 3  │               │
│  │ (AZ-1)       │  │ (AZ-2)       │  │ (AZ-3)       │               │
│  │              │  │              │  │              │               │
│  │ Hot tier:    │  │ Hot tier:    │  │ Hot tier:    │               │
│  │ SSD, 500GB   │  │ SSD, 500GB   │  │ SSD, 500GB   │               │
│  │ 32GB RAM     │  │ 32GB RAM     │  │ 32GB RAM     │               │
│  │ JVM heap: 16G│  │ JVM heap: 16G│  │ JVM heap: 16G│               │
│  └──────────────┘  └──────────────┘  └──────────────┘               │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │ Warm Tier (optional, added as data grows):               │       │
│  │ 2 warm data nodes, HDD, 2TB, force-merged read-only indices │     │
│  └──────────────────────────────────────────────────────────┘       │
│                                                                      │
│  Index Aliases:                                                      │
│  • transactions-current → transactions-2026-04 (active)              │
│  • transactions-search  → transactions-2026-*  (search all)          │
│                                                                      │
│  Schema Registry: Confluent Schema Registry                          │
│  (Avro schemas for Kafka events, compatible with ES index mapping)   │
└──────────────────────────────────────────────────────────────────────┘
```

6.2 Node Sizing Rationale

Component	Size	Reasoning
Data nodes	3 x r6g.xlarge (4 vCPU, 32GB RAM)	Each shard ~50M docs. 6 shards x 1 replica = 12 shard copies across 3 nodes = 4 shards per node. 16GB JVM heap (50% of RAM, ES best practice).
Master nodes	3 x c6g.large (2 vCPU, 4GB)	Dedicated masters prevent cluster instability during heavy indexing. Lightweight — only manage cluster state.
Storage	SSD (gp3), 500GB per data node	Transaction documents are small (~500 bytes each). 100M docs ≈ 50GB indexed. 500GB provides headroom for replicas and ILM.

6.3 Index Lifecycle Management

```
┌──────────┐      ┌──────────┐      ┌──────────┐      ┌──────────┐
│   HOT    │──────►│  WARM    │──────►│   COLD   │──────►│  DELETE  │
│          │      │          │      │          │      │          │
│ 0-3 mo   │      │ 3-12 mo  │      │ 12-36 mo │      │ > 36 mo  │
│ SSD      │      │ HDD      │      │ Frozen   │      │ Purged   │
│ Active   │      │ Read-only│      │ Snapshot │      │          │
│ indexing │      │ Force-   │      │ to S3    │      │ PG is    │
│          │      │ merged   │      │ Slow but │      │ SoT for  │
│          │      │          │      │ searchable│     │ compliance│
└──────────┘      └──────────┘      └──────────┘      └──────────┘
```

7. When to Consider Alternatives in the Future

Trigger	Alternative	Reasoning
ES licensing becomes prohibitive (SSPL concerns)	**OpenSearch**	Drop-in replacement, Apache 2.0 license. 98% API compatible.
Need AI-powered semantic search ("find transactions similar to my rent payments")	**Elasticsearch with vector search** or **Pinecone/Weaviate**	ES 8.x supports `dense_vector` + kNN. For dedicated vector search, Pinecone/Weaviate are purpose-built.
Simplify ops for a smaller team	**Managed Elastic Cloud** or **AWS OpenSearch Serverless**	Removes node management, auto-scaling, patching. Higher cost but lower ops burden.
Real-time analytics becomes a primary use case	**Apache Druid** or **ClickHouse** alongside ES	ES aggregations work for dashboards but aren't optimized for heavy OLAP. Druid/ClickHouse are purpose-built for real-time analytics at scale.

End of Document — Search Engine Choices v1.0